

Asynchronous Disposable in C# 8 with Examples

In this article, I am going to discuss **Asynchronous Disposable in C# 8** with Examples. Please read our previous article where we discussed [Asynchronous Streams in C# 8](#) with Examples. Starting with C# 8.0, the language supports asynchronous disposable types that implement the `IAsyncDisposable` interface. You use the `await using` statement to work with an asynchronously disposable object.

Implement the `DisposeAsync` Method of the `IAsyncDisposable` Interface in C#

The `IAsyncDisposable` interface was introduced as part of C# 8.0. We need to implement the `DisposeAsync()` method when we want to perform resource clean-up in the same way as we used to by implementing the `Dispose()` method of `IDisposable` Interface. The difference between these two interfaces is that the `DisposeAsync()` method allows for asynchronous clean-up operations whereas the `Dispose()` method performs synchronous clean-up operations. The `DisposeAsync()` method of the `IAsyncDisposable` interface returns a `ValueTask` that represents the asynchronous dispose of operation.

The point that you need to keep in mind is that when we are implementing the `IAsyncDisposable` (for asynchronous clean-up) interface and then we also need to make sure that the class will also implement the `IDisposable` (for synchronous clean-up) interface. The reason is that a good implementation pattern of the `IAsyncDisposable` interface is to be prepared for both synchronous and asynchronous dispose. Before proceeding further, I assume that you are already familiar with how to implement a `Dispose` method of the `IDisposable` interface for synchronous clean-up.

The `DisposeAsync()` method of the `IAsyncDisposable` Interface in C#

The public parameterless `DisposeAsync()` method is called automatically in an `await using` statement, and the purpose of this `DisposeAsync()` method is to free the unmanaged resources. Freeing the memory associated with a managed object is always the responsibility of the garbage collector. Because of this, it has a standard implementation as follows:

```
public async ValueTask DisposeAsync()
{
    // Perform async clean-up.
    await DisposeAsyncCore();
    // Dispose of unmanaged resources.
    Dispose(false);
    // Dispose methods should call SuppressFinalize
    // Suppress finalization.
    GC.SuppressFinalize(this);
}
```

Note: The difference between the Async Dispose Pattern and Dispose Pattern is that the call from `DisposeAsync()` to the `Dispose(bool)` overload method is given `false` as an argument. When implementing the `Dispose()` method, however, `true` is passed instead. This helps ensure functional equivalence with the synchronous dispose of pattern, and further ensures that finalizer code paths still get invoked. In other words, the `DisposeAsyncCore()` method will dispose of managed resources asynchronously, so you don't want to dispose of them synchronously as well. Therefore, call `Dispose(false)` instead of `Dispose(true)`.

The `DisposeAsyncCore()` Method

The `DisposeAsyncCore()` method is intended to perform the asynchronous clean-up of managed resources. It encapsulates the common asynchronous clean-up operations when a subclass inherits a base class that is an implementation of `IAsyncDisposable`. The `DisposeAsyncCore()` method is virtual so that derived classes can define additional clean-up by overriding this method. If an implementation of `IAsyncDisposable` is sealed, the `DisposeAsyncCore()` method is not needed, and the asynchronous clean-up can be performed directly in the `DisposeAsync()` method.

Any non-sealed class should have an additional `DisposeAsyncCore()` method that should return a `ValueTask`. So, the class should have a public `IAsyncDisposable.DisposeAsync()` implementation that has no parameters as well as a protected virtual `ValueTask DisposeAsyncCore()` method with the following:

```
protected virtual ValueTask DisposeAsyncCore()
{
}
```

Implementing Asynchronous Disposable in C# 8 with Examples

Let us understand this with examples. The following example shows the simplest way to implement `IAsyncDisposable`. In the below example, we created a class called `Sample` and this class implements the `IAsyncDisposable` interface and provides the implementation for the `DisposeAsync` method. This `DisposeAsync` method takes the responsibility to clean up the memory asynchronously. As the `DisposeAsync` method is a static method, so it requires an await operation inside it and hence we have used the `await Task.Delay(1000)` to delay the operation for 1 millisecond. Again, let us assume that we are using `TextWriter` as an unmanaged resource. The `DisposeAsync` is implicitly called from the main method at the end of the await using statement block.

```
using System;
using System.Threading.Tasks;
using System.IO;
namespace Csharp8Features
{
    class AsynchronousDisposable
    {
        static async Task Main(string[] args)
        {
            await using (var disposableObject = new Sample())
            {
                Console.WriteLine("Welcome to C#.NET");
            } // DisposeAsync method called implicitly
            Console.WriteLine("Main Method End");
        }
    }
    public class Sample : IAsyncDisposable
    {
        static readonly string filePath = @"D:\MyTextFile1.txt";
        private TextWriter? textWriter = File.CreateText(filePath);
        public async ValueTask DisposeAsync()
        {
            if (textWriter != null)
            {
                textWriter = null;
            }
        }
    }
}
```

```

    }

    await Task.Delay(1000);
    Console.WriteLine("DisposeAsync Clean-up the Memory!");
}
}
}

```

Now, run the above code and you should get the following output.

```

Welcome to C#.NET
DisposeAsync Clean-up the Memory!
Main Method End

```

Note: To properly consume an object that implements the `IAsyncDisposable` interface, you use the `await` and `using` keywords together.

Creating Virtual `DisposeAsyncCore` Method:

According to the MS guidelines (<https://docs.microsoft.com/en-us/dotnet/standard/garbage-collection/implementing-disposeasync>), a non-sealed class should always be considered inheritable. Similar to the inheritance pattern for a regular synchronous `IDisposable` implementation, we should also add a virtual method to override the disposal behavior in the subclass.

All non-sealed classes should be considered as a potential base class because they could be inherited. If we want to implement the Asynchronous Disposable pattern for any non-sealed class, then we must provide the protected virtual `DisposeAsyncCore()` method.

For a better understanding, please have a look at the below code. As you can see in the below code, the `Sample` class is a non-sealed class and implements the `IAsyncDisposable` interface and provide an implementation for the `DisposeAsync` method. Here, we have also provided one Virtual `DisposeAsyncCore` method and this method can be overridden by a child class. Further if you notice, from the `DisposeAsync` method we are calling the `DisposeAsyncCore` method.

```

public class Sample : IAsyncDisposable
{
    public async ValueTask DisposeAsync()
    {
        Console.WriteLine("Delaying!");
        await Task.Delay(1000);
        Console.WriteLine("Disposed!");
    }
    protected virtual async ValueTask DisposeAsyncCore()
    {
        Console.WriteLine("DisposeAsyncCore Delaying!");
        await Task.Delay(1000);
        Console.WriteLine("DisposeAsyncCore Disposed!");
    }
}

```

Complete Example Code:

```
using System;
using System.Threading.Tasks;
using System.IO;
namespace Csharp8Features
{
    class AsynchronousDisposable
    {
        static async Task Main(string[] args)
        {
            await using (var disposableObject = new Sample())
            {
                Console.WriteLine("Welcome to C#.NET");
            } // DisposeAsync method called implicitly
            Console.WriteLine("Main Method End");
        }
    }
    public class Sample : IAsyncDisposable
    {
        static readonly string filePath = @"D:\MyTextFile1.txt";
        private TextWriter? textWriter = File.CreateText(filePath);
        public async ValueTask DisposeAsync()
        {
            await DisposeAsyncCore().ConfigureAwait(false);
            Console.WriteLine("DisposeAsync Clean-up the Memory!");
        }
        protected virtual async ValueTask DisposeAsyncCore()
        {
            if (textWriter != null)
            {
                await textWriter.DisposeAsync().ConfigureAwait(false);
            }
            textWriter = null;
            Console.WriteLine("Virtual DisposeAsyncCore Clean-up the Memory");
        }
    }
}
```

Now, run the above code and you should get the following output.

```
Welcome to C#.NET
Virtual DisposeAsyncCore Clean-up the Memory
DisposeAsync Clean-up the Memory!
Main Method End
```

Overriding the DisposeAsyncCore Method in Child Class:

Now, let us create a subclass with the name SampleInherited by inheriting from the Sample class and also override the DisposeAsyncCore virtual method as follows:

```
public class SampleInherited : Sample
{
    protected override async ValueTask DisposeAsyncCore()
    {
        await base.DisposeAsyncCore();
        Console.WriteLine("DisposeAsyncCore Subclass Delaying!");
        await Task.Delay(1000);
        Console.WriteLine("DisposeAsyncCore Subclass Disposed!");
    }
}
```

Next, from the main method, now, instead of creating an instance Sample class, create an instance of SampleInherited as follows:

```
static async Task Main(string[] args)
{
    await using (var disposableObject = new SampleInherited())
    {
        Console.WriteLine("Welcome to C#.NET");
    } // DisposeAsync method called implicitly
    Console.WriteLine("Main Method End");
}
```

Now, when we reach the end of the await using block, first, it will call the DisposeAsync method of the Sample class. From the DisposeAsync method of the Sample class, it will call the DisposeAsyncCore method of the subclass and from the subclass DisposeAsyncCore method, it will call the superclass DisposeAsyncCore method. If you run the application in debugging mode, then you can see this. The following is a complete example.

```
using System;
using System.Threading.Tasks;
using System.IO;
namespace Csharp8Features
{
    class AsynchronousDisposable
    {
        static async Task Main(string[] args)
        {
            await using (var disposableObject = new SampleInherited())
            {
                Console.WriteLine("Welcome to C#.NET");
            } // DisposeAsync method called implicitly
            Console.WriteLine("Main Method End");
        }
    }
    public class Sample : IAsyncDisposable
    {
        static readonly string filePath = @"D:\MyTextFile1.txt";
```

```

private TextWriter? textWriter = File.CreateText(filePath);
public async ValueTask DisposeAsync()
{
    await DisposeAsyncCore().ConfigureAwait(false);
    Console.WriteLine("DisposeAsync Clean-up the Memory!");
}
protected virtual async ValueTask DisposeAsyncCore()
{
    if (textWriter != null)
    {
        await textWriter.DisposeAsync().ConfigureAwait(false);
    }
    textWriter = null;
    Console.WriteLine("Virtual DisposeAsyncCore Clean-up the Memory");
}
}
public class SampleInherited : Sample
{
    protected override async ValueTask DisposeAsyncCore()
    {
        await base.DisposeAsyncCore();
        Console.WriteLine("Subclass DisposeAsyncCore Clean-up the Memory");
    }
}
}

```

Output:

```

Welcome to C#.NET
Virtual DisposeAsyncCore Clean-up the Memory
Subclass DisposeAsyncCore Clean-up the Memory
DisposeAsync Clean-up the Memory!
Main Method End

```

Implementing both Dispose and Async Dispose Patterns in C#:

You may need to implement both the `IDisposable` and `IAsyncDisposable` interfaces, especially when your class scope contains instances of these implementations. Doing so ensures that you can properly cascade clean-up calls.

Another important recommendation from Microsoft is that we should need to implement both the `IDisposable` and `IAsyncDisposable` interfaces in the class since not every consumer might be able to properly handle the new style yet. For example, a lot of older Inversion of Control frameworks are not capable of handling asynchronous disposals yet. Doing so ensures that you can properly cascade clean-up calls. Let us see an example, that implements both interfaces and demonstrates the proper guidance for clean-up. We are using the same `Sample` class.

```

public class Sample : IDisposable, IAsyncDisposable
{

```

```

private Stream? disposableResource = new MemoryStream();
private Stream? asyncDisposableResource = new MemoryStream();
public void Dispose()
{
    GC.SuppressFinalize(this);
    Console.WriteLine("Dispose Clean-up the Memory!");
}
public async ValueTask DisposeAsync()
{
    await DisposeAsyncCore().ConfigureAwait(false);
    Dispose();
    GC.SuppressFinalize(this);
    Console.WriteLine("DisposeAsync Clean-up the Memory!");
}
protected virtual async ValueTask DisposeAsyncCore()
{
    if (asyncDisposableResource != null)
    {
        await asyncDisposableResource.DisposeAsync().ConfigureAwait(false);
    }
    if (disposableResource is IAsyncDisposable disposable)
    {
        await disposable.DisposeAsync().ConfigureAwait(false);
    }
    else
    {
        disposableResource?.Dispose();
    }
    asyncDisposableResource = null;
    disposableResource = null;
    Console.WriteLine("Virtual DisposeAsyncCore Clean-up the Memory");
}
}

```

This way we can ensure that our resources can be cleaned up properly in both fashions. Here, we are using the Stream class which provides both synchronous as well as asynchronous disposal. If you go to the definition of Stream class, then you will see that it implements both IDisposable and IAsyncDisposable interfaces as shown in the below image.

```

//
// Summary:
//     Provides a generic view of a sequence of bytes. This is an abstract class.
public abstract class Stream : MarshalByRefObject, IAsyncDisposable, IDisposable

```

Complete Example Code:

```

using System;
using System.Threading.Tasks;

```

```

using System.IO;
namespace Csharp8Features
{
    class AsynchronousDisposable
    {
        static async Task Main(string[] args)
        {
            await using (var disposableObject = new Sample())
            {
                Console.WriteLine("Welcome to C#.NET");
            } // DisposeAsync method called implicitly
            Console.WriteLine("Main Method End");
        }
    }
    public class Sample : IDisposable, IAsyncDisposable
    {
        private Stream? disposableResource = new MemoryStream();
        private Stream? asyncDisposableResource = new MemoryStream();
        public void Dispose()
        {
            GC.SuppressFinalize(this);
            Console.WriteLine("Dispose Clean-up the Memory!");
        }
        public async ValueTask DisposeAsync()
        {
            await DisposeAsyncCore().ConfigureAwait(false);
            Dispose();
            GC.SuppressFinalize(this);
            Console.WriteLine("DisposeAsync Clean-up the Memory!");
        }
        protected virtual async ValueTask DisposeAsyncCore()
        {
            if (asyncDisposableResource != null)
            {
                await asyncDisposableResource.DisposeAsync().ConfigureAwait(false);
            }
            if (disposableResource is IAsyncDisposable disposable)
            {
                await disposable.DisposeAsync().ConfigureAwait(false);
            }
            else
            {
                disposableResource?.Dispose();
            }
            asyncDisposableResource = null;
            disposableResource = null;
            Console.WriteLine("Virtual DisposeAsyncCore Clean-up the Memory");
        }
    }
}

```

Output:

```
Welcome to C#.NET
Virtual DisposeAsyncCore Clean-up the Memory
Dispose Clean-up the Memory!
DisposeAsync Clean-up the Memory!
Main Method End
```

In the next article, I am going to discuss [Indices and Ranges in C# 8](#) with Examples. Here, in this article, I try to explain **Asynchronous Disposable in C# 8** with Examples. I hope you enjoy this Asynchronous Disposable in C# 8 with Examples article.